SOFTWARE REQUIREMENTS SPECIFICATION

Cinepool Jaipur

Implemented system requirements, interfaces, data relationships, controls and verification checks

Version 1.0 | 28 October 2026

Source baseline: master at bea5ca0

Current interface name: Picturehouse Jaipur

Prepared from the implementation and the documented local MySQL inspection. Current behavior and proposed work are identified separately.

1. Introduction

1.1 Purpose

This SRS records the observable software contract of the current Cinepool Jaipur project. The interface and Java source call it Picturehouse Jaipur. “Shall” statements in sections 3-7 describe the current implementation unless marked **Proposed**. Acceptance checks specify how a reviewer could confirm behavior; they are not claims that an automated test was run for this document.

1.2 Evidence and uncertainty

Requirements were traced to the repository's React source, Spring controllers, configuration, JPA entities, repositories and Maven/npm manifests. The database inventory was inspected locally for the earlier verified project report on 27 October 2026: MySQL schema `picturehouse`, 15 base tables, all InnoDB. That snapshot is evidence of one local installation, not a guarantee of the table count on another machine. The application uses `ddl-auto: update`, so physical schema details may evolve with future code. No runtime load, security penetration, accessibility, or production payment test was performed for this SRS.

1.3 Terms

- **Clerk subject:** signed-in user's identifier in a validated JWT.
- **On sale:** the boolean showtime flag used by catalog and owner queries.
- **Provider order:** Razorpay test order created on the backend and stored in `razorpay_orders`.
- **Ticket booking:** confirmed current-flow booking stored in `ticket_bookings`, separate from legacy `bookings`.
- **Ride request:** a request to join a `ride_posts` entry; it is not a ticket booking.
- **Conditional:** implemented code that needs configuration or provider availability to work.

2. System description

2.1 Architecture and deployment

The browser runs a React 19 client built with Vite 8. It calls a Java 21 / Spring Boot 3.5.6 HTTP API on port 8080. Spring Data JPA/Hibernate persists entities to MySQL. Clerk handles interactive authentication; the API validates Clerk JWTs using configured issuer and JWKS. Razorpay Test Mode handles payment checkout through backend order and verification endpoints. Spring Mail can send ticket emails if SMTP is configured. ZXing generates QR image data for ticket responses. The Vite development server normally runs on port 5173. H2 and PostgreSQL drivers are dependencies in Maven, but the checked-in default datasource is MySQL.

Context flow: Browser -> Spring API -> MySQL; Browser -> Clerk; Browser -> Razorpay Checkout; Spring API -> Razorpay API; Spring API -> SMTP server when configured.

2.2 Actors and authorization

Actor	API access
Visitor	`GET /catalog`, `GET /catalog/showtimes/{id}/seats`, `GET /api/rides?date=...`, health endpoint.
Authenticated user	Ticket, test payment, ride mutation/conversation and owner-access request endpoints.
Approved owner	Dashboard and theatre-scoped movie/showtime management after membership check.
Configured administrator	Pending owner requests and approve operation when JWT subject is in `ADMIN_CLERK_USER_IDS`.

The `SecurityConfig` permits `/catalog` through its general rule and marks only `GET /api/rides` explicitly public under `/api/**`. Backend authorization for owner operations is enforced in the controller, not inferred from a frontend tab.

3. Functional requirements

3.1 Catalog and show selection

SRS-CAT-01 (Implemented): `GET /catalog` shall return active movies and active Jaipur theatres with on-sale shows whose start is at or after the current Asia/Kolkata time and earlier than the start of the day eight days from today. The response shall include a development-data notice. Evidence: `CatalogController.getCatalog`.

SRS-CAT-02 (Implemented): the browser shall offer today and the next seven calendar dates and filter visible show buttons by selected date and movie. A movie modal currently displays at most the first three matching shows per theatre (`slice(0, 3)`); the theatre list can show all matching buttons. Evidence: `src/App.jsx`.

SRS-CAT-03 (Implemented): the browser shall refresh the catalog every 15 seconds and when the window becomes focused or visible. It shall show a load error when the API is unavailable. Evidence: `src/App.jsx`.

SRS-CAT-04 (Implemented): `GET /catalog/showtimes/{id}/seats` shall return ordered seat codes and booked flags for an existing showtime; an unknown ID shall return 404. The seat modal shall poll this endpoint about every 2.5 seconds while open. Evidence: `CatalogController.getSeats`, `src/App.jsx`.

SRS-CAT-05 (Implemented): search shall filter movie titles case-insensitively in the browser. Theatre-name filtering is not implemented, despite the current search placeholder. Evidence: `src/App.jsx`.

Acceptance checks: compare a past and future on-sale show against the API response; change the date and movie and inspect the listed shows; request an unknown seat-map ID; enter part of a movie title in search.

3.2 Seat pricing, payment and ticket creation

SRS-PAY-01 (Conditional): the server shall accept one to eight distinct seat codes matching `A1` through `F8` for an on-sale show. It shall reject absent, duplicate, invalid or already booked seats before creating an order. Evidence: `RazorpayController.createOrder`.

SRS-PAY-02 (Implemented rule): the server shall calculate `amountPaise` from codes: rows A-C = 25,000 paise, D-E = 30,000 paise, F = 50,000 paise. The client-provided display total shall not determine the provider order amount. Evidence: `RazorpayController.priceForSeatPaise`.

SRS-PAY-03 (Conditional): with configured Razorpay test credentials, `POST /api/payments/razorpay/orders` shall create a provider order and persist its order ID, user subject, amount, showtime ID and seat list. If test

credentials are missing, it shall return 503. Evidence: `RazorpayController.createOrder` and `requireTestCredentials`.

SRS-PAY-04 (Conditional): `POST /api/payments/razorpay/verify` shall require that the saved order belongs to the signed-in user, the checkout HMAC signature matches, and the provider lookup reports the same order ID, amount, INR currency and `captured` status. It shall create no ticket when these checks fail. Evidence: `RazorpayController.verifyPayment`, `RazorpayClient`.

SRS-PAY-05 (Implemented rule): after provider confirmation, a transaction shall lock selected seat rows, mark free seats booked, update the order, and create a `ticket_bookings` record. If the seats are unavailable, the server shall try to refund the captured payment instead of creating a ticket. A refund failure shall surface a support-directed conflict response. Evidence: `RazorpayController.verifyPayment`, `ShowtimeSeatRepository.lockSeats`.

SRS-PAY-06 (Implemented rule): repeated verification of an order with an existing booking for that user shall return that booking rather than creating another. Evidence: `RazorpayController.verifyPayment`.

Acceptance checks: inspect provider order amount for each fare row; submit invalid or duplicate codes; try a wrong signature; verify the same captured order twice; simulate a captured seat conflict in a controlled test environment. These checks require sandbox keys and must not be performed against live money.

3.3 Tickets and email

SRS-TKT-01 (Implemented): `GET /api/bookings/mine` shall return only tickets for the JWT subject, newest first, with booking reference, movie, theatre, show, seats, amount, created time, QR data URL and email-sent flag. Evidence: `TicketController.myTickets`.

SRS-TKT-02 (Conditional): `POST /api/bookings/{reference}/email` shall find a ticket belonging to the JWT subject and send it to the token's `email` claim when SMTP is configured. Missing claim shall return 412; unavailable mail configuration shall return 503. Evidence: `TicketController.emailTicket`, `TicketEmailService`.

SRS-TKT-03 (Implemented): the ticket QR shall be generated from a stored unique ticket code. A gate validation or one-time redemption endpoint is not present. Evidence: `TicketController.qrDataUrl`, `TicketBooking`.

Acceptance checks: compare two signed-in accounts; request an unknown or another user's reference; inspect the returned QR and email flag; attempt email with and without the required configuration.

3.4 Ride sharing and chat

SRS-RID-01 (Implemented): `GET /api/rides?date=YYYY-MM-DD` shall list active rides for that date in departure order. It is a public API route. Evidence: `RideController.list`, `SecurityConfig`.

SRS-RID-02 (Implemented): signed-in users shall be able to post a ride dated today through 90 days ahead, with a future departure on the same date, 1-6 available seats, and INR 1-10,000 contribution per person. Pickup, theatre, movie and showtime are stored as text. Evidence: `RideController.post`, `RidePost`.

SRS-RID-03 (Implemented): a signed-in non-owner shall be able to request to join an active ride with at least one open seat. One request per ride and requester is protected by a database unique key. Evidence: `RideController.requestToJoin`, `RideJoinRequest`.

SRS-RID-04 (Implemented): the ride owner and requester shall be able to list their conversations and messages. Only these participants may read or write a conversation. A declined request shall reject new messages. Message length is limited to 1-1,200 characters after validation. Evidence: `RideController.conversations`, `participantRequest`, `sendMessage`.

SRS-RID-05 (Implemented): only the ride owner shall accept or decline a pending request. Acceptance shall lock request and ride rows, reserve one seat, and change status to accepted; a second decision shall return conflict. Evidence: `RideController.accept` and `decline`, repository locks.

Acceptance checks: post invalid dates/seats/fares; request one's own ride; read a third party conversation; accept the same request twice; check remaining seats after one acceptance.

3.5 Theatre partner and administration

SRS-OWN-01 (Implemented): `POST /api/owner/access-requests` shall create or return the current user's membership for the named theatre. Its initial approved flag is false. `GET /api/owner/access` shall return the user's memberships and status. Evidence: `OwnerPortalController.requestAccess` and `access`.

SRS-OWN-02 (Implemented): only a JWT subject in `ADMIN_CLERK_USER_IDS` shall read pending requests or approve a membership. Evidence: `OwnerPortalController.requireAdmin`, `pending`, `approve`.

SRS-OWN-03 (Implemented): an owner without an approved theatre shall receive 403 from the dashboard/management options; operations targeting a theatre shall verify approved membership for that exact theatre. Evidence: `OwnerPortalController.approvedTheatreIds`, `requireTheatreAccess`.

SRS-OWN-04 (Implemented): an approved owner shall see their approved theatre list and on-sale shows, with ticket count and gross sales computed from ticket bookings for each show. Evidence: `OwnerPortalController.dashboard`.

SRS-OWN-05 (Implemented): an approved owner shall be able to add a movie assignment to their theatre. Updating details is allowed only when the movie is not shared with another theatre. Request fields require nonblank title, language, genre, certification and synopsis; title length at most 120, duration at least 1 minute. Evidence: `OwnerPortalController.addMovie`, `updateMovie`, `MovieRequest`.

SRS-OWN-06 (Implemented): an approved owner shall be able to create a future show within 90 days on a screen in their theatre. Creation shall persist 48 seat rows A1-F8. The movie must be active; creation can assign it to the theatre if needed. Evidence: `OwnerPortalController.addShowtime`.

SRS-OWN-07 (Implemented limitation): update of an existing show shall require access to its current theatre and a future start time. The current method saves only the new start time and format; it validates but does not switch screen and does not change movie. Evidence: `OwnerPortalController.updateShowtime`, `Showtime.reschedule`.

Acceptance checks: attempt management with pending approval; attempt another theatre's ID; approve as a non-admin; update a shared movie; create a valid show and inspect 48 seats; edit a show and inspect its unchanged movie/screen.

4. External interfaces and API inventory

All routes below are relative to the Spring API host, normally `http://localhost:8080`. JSON is used for request and response bodies. Bearer JWT authentication is required where the security rule says protected; route-specific checks may be stricter.

Route	Auth	Purpose
`GET /catalog`	Public	Active movie and Jaipur show catalog.
`GET /catalog/showtimes/{id}/seats`	Public	Seat codes and booked flags.
`GET /api/auth/me`	JWT	Basic token identity.
`POST /api/payments/razorpay/orders`	JWT	Create priced test order.
`POST /api/payments/razorpay/verify`	JWT	Verify captured test payment and reserve seats.

Route	Auth	Purpose
`GET /api/bookings/mine`	JWT	Purchaser's tickets.
`POST /api/bookings/{reference}/email`	JWT	Email own ticket.
`GET /api/rides?date=...`	Public	Active rides on date.
`POST /api/rides`	JWT	Post ride.
`POST /api/rides/{rideId}/requests`	JWT	Request ride seat.
`GET /api/rides/conversations`	JWT	Own ride conversations.
`GET/POST /api/rides/conversations/{id}/messages`	JWT + participant	Read or send chat.
`PUT /api/rides/conversations/{id}/accept`	JWT + ride owner	Accept request.
`PUT /api/rides/conversations/{id}/decline`	JWT + ride owner	Decline request.
`GET /api/owner/access`	JWT	Memberships.
`POST /api/owner/access-requests`	JWT	Request theatre access.
`GET /api/owner/dashboard`	JWT + approved owner	Sales.
`GET /api/owner/manage-options`	JWT + approved owner	Owned theatres, screens, movies.
`POST /api/owner/theatres/{id}/movies`	JWT + approved owner	Add movie assignment.
`PUT /api/owner/theatres/{id}/movies/{movieId}`	JWT + approved owner	Edit eligible movie.
`POST /api/owner/theatres/{id}/showtimes`	JWT + approved owner	Add show and seats.
`PUT /api/owner/showtimes/{id}`	JWT + approved owner	Edit show start and format.
`GET /api/owner/admin/requests`	JWT + configured admin	Pending requests.
`PUT /api/owner/admin/requests/{id}/approve`	JWT + configured admin	Approve membership.

These endpoints are sourced from controller route annotations. No DELETE owner, ride, or ticket operation is present in these controllers. Controller errors use standard HTTP codes including 400, 403, 404, 409, 412, 502 and 503 depending on the case; exact response text may change.

5. Data requirements and relationships

5.1 Physical MySQL schema snapshot

The observed database name is `picturehouse`. The 15 observed base tables are:

1. Catalog: `movies`, `theatres`, `screens`, `showtimes`, `showtime_seats`, `theatre_movies`.
2. Current ticket flow: `razorpay_orders`, `ticket_bookings`.
3. Current ride flow: `ride_posts`, `ride_join_requests`, `ride_chat_messages`.
4. Partner access: `theatre_owner_memberships`.
5. Legacy entity tables with no active controller workflow in this baseline: `accounts`, `bookings`, `carpool_rides`.

All 15 observed tables had primary keys. The inspected schema had 13 foreign keys and 12 unique constraints. It had no database triggers, views, stored routines or `CHECK` constraints. Those numbers describe the observed local schema, not a migration specification.

5.2 Database-enforced foreign-key relationships

Child	Parent	Relationship
`screens.theatre_id`	`theatres.id`	Screen belongs to theatre.
`showtimes.movie_id`	`movies.id`	Show runs a movie.
`showtimes.screen_id`	`screens.id`	Show occurs on a screen.
`showtime_seats.showtime_id`	`showtimes.id`	Seat state belongs to show.
`theatre_movies.theatre_id`	`theatres.id`	Theatre/movie assignment.
`theatre_movies.movie_id`	`movies.id`	Theatre/movie assignment.
`theatre_owner_memberships.theatre_id`	`theatres.id`	Owner access targets a theatre.
`ride_join_requests.ride_post_id`	`ride_posts.id`	Passenger request belongs to ride.
`ride_chat_messages.join_request_id`	`ride_join_requests.id`	Chat belongs to request.
`bookings.customer_id`	`accounts.id`	Legacy booking/customer.
`bookings.showtime_id`	`showtimes.id`	Legacy booking/show.
`carpool_rides.driver_id`	`accounts.id`	Legacy ride/driver.
`carpool_rides.destination_showtime_id`	`showtimes.id`	Legacy ride/show.

The current `razorpay_orders.showtime_id`, `ticket_bookings.provider_order_id`, ride labels, and Clerk user IDs are application-level references or copied values; they are **not** among the 13 observed foreign keys. The simplified relationship picture is: `theatres -> screens -> showtimes -> showtime_seats`, `movies -> showtimes`, `theatres <-> movies` through `theatre_movies`, `theatres -> theatre_owner_memberships`, and `ride_posts -> ride_join_requests -> ride_chat_messages`.

5.3 Key uniqueness and transactions

The observed schema has unique keys for one screen name per theatre, one seat code per show, one movie assignment per theatre, one owner membership per Clerk user/theatre, and one ride request per ride/requester. Provider order ID, booking reference and ticket code are unique in current ticket tables. Foreign keys do not by themselves enforce every business rule. Seat booking and ride acceptance use Spring transactions plus pessimistic repository locks. MySQL's InnoDB engine was observed, but no application-level transaction isolation or deadlock retry policy is explicitly configured.

6. Security and quality requirements

6.1 Implemented controls

SRS-SEC-01: protected `/api/**` endpoints shall require a valid bearer JWT, except the explicitly public ride GET route. The JWT decoder checks issuer/time and checks `azp` against configured origins when an `azp` claim is present. An audience claim requirement is not visible in `SecurityConfig`.

SRS-SEC-02: owner mutations shall check approved membership on the server, and admin approval shall compare the JWT subject to configured admin IDs. Ride chat shall enforce participant membership, and ticket retrieval/email shall scope to the current subject.

SRS-SEC-03: the API shall use configured exact CORS origins and stateless Spring Security sessions. The security configuration disables CSRF for the bearer-token API. The repository's `.env` files and local database file are ignored by Git; `.env.example` remains a template and must not contain real credentials.

SRS-SEC-04: test payment verification shall check checkout HMAC and a provider payment lookup before creating a ticket. This does not constitute a production webhook or reconciliation mechanism.

6.2 Quality characteristics and evidence limits

- **Consistency:** transactions and row locks protect the current seat and ride-acceptance write paths; no broad concurrency benchmark was run for this document.
- **Persistence:** JPA writes current catalog, ticket, ride, and membership entities to MySQL when datasource credentials are valid. Schema generation uses `ddl-auto: update`, suitable for this prototype but not a controlled migration history.
- **Freshness:** the browser polls catalog every 15 seconds and open seat maps every 2.5 seconds. There is no real-time delivery guarantee; WebSocket configuration exists but the current UI uses polling for these flows.
- **Availability, performance, accessibility, backup, privacy retention:** no measured targets or completed audit are present in the provided source. Treat these as **Not verified**, not as satisfied quality attributes.
- **Time handling:** catalog filtering explicitly uses Asia/Kolkata. Other controller checks use default JVM local time; consistency across differently configured hosts is not guaranteed by one shared zone setting.

7. Configuration and operational constraints

Setting	Current role
`DATABASE_URL`, `DATABASE_USER`, `DATABASE_PASSWORD`, `DATABASE_DRIVER`	MySQL datasource; default URL is `jdbc:mysql://localhost:3306/picturehouse?serverTimezone=UTC`. Password must be supplied.
`CLERK_ISSUER_URL`, `CLERK_JWKS_URL`, `CLERK_ALLOWED_ORIGINS`	Backend JWT verification; issuer and JWKS are required to start the API.
`VITE_CLERK_PUBLISHABLE_KEY`	Frontend Clerk sign-in configuration.
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Backend Razorpay Test Mode.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `TICKET_EMAIL_FROM`	Optional ticket email.
`ADMIN_CLERK_USER_IDS`	Comma-separated Clerk subjects allowed to approve owner access.
`APP_CORS_ALLOWED_ORIGINS`	Allowed browser origins for the API.

Do not commit actual passwords or provider secrets. When the backend is unavailable, the frontend shows a catalog error and no booking can complete. When the database is empty, seed behavior may add development examples; it does not keep generating a rolling schedule after that initial condition.

8. Known limitations and proposed requirements

The following are **not satisfied in the current baseline**. They are candidate requirements for a future release:

- **SRS-FUT-01 (Proposed):** implement production provider webhooks, event idempotency and payment reconciliation before live checkout.
- **SRS-FUT-02 (Proposed):** define a seat-hold/expiry policy and implement recovery when a customer abandons checkout.
- **SRS-FUT-03 (Proposed):** validate and redeem QR tickets at a theatre gate with authorized staff and an auditable state change.
- **SRS-FUT-04 (Proposed):** import verified theatre schedules and real screen layouts; replace development catalog records and JPA auto-update with reviewed migrations.
- **SRS-FUT-05 (Proposed):** link ride posts to a showtime ID and add safety, reporting and moderation requirements.
- **SRS-FUT-06 (Proposed):** align the show editor UI with backend-supported changes, refresh the date picker across midnight, and either implement theatre search or label the field as movie search.
- **SRS-FUT-07 (Proposed):** define measurable response-time, uptime, accessibility and data-retention criteria and verify them before production release.

9. Traceability summary

Product goal	Implementation requirement IDs	Principal source
G1 Discovery	CAT-01 through CAT-05	`CatalogController.java`, `src/App.jsx`
G2 Booking	CAT-04, PAY-01 through PAY-06	`RazorpayController.java`, seat repository
G3 Retrieval	TKT-01 through TKT-03	`TicketController.java`, email service
G4 Shared travel	RID-01 through RID-05	`RideController.java`, ride entities
G5 Partner control	OWN-01 through OWN-07, SEC-02	`OwnerPortalController.java`

10. Source index and document control

The authoritative repository sources for this SRS are `src/App.jsx`, `src/RideBoard.jsx`, `src/OwnerPortal.jsx`, `src/ClerkAuth.jsx`, `backend/src/main/java/in/picturehouse/jaipur/api/`, `backend/src/main/java/in/picturehouse/jaipur/config/SecurityConfig.java`, `backend/src/main/java/in/picturehouse/jaipur/domain/`, `backend/src/main/java/in/picturehouse/jaipur/repository/`, `backend/src/main/java/in/picturehouse/jaipur/integration/`, `backend/src/main/resources/application.yml`, `package.json`, and `backend/pom.xml`. The local MySQL inspection is documented in `output/pdf/Cinepool_Jaipur_Verified_Project_Report.pdf` and its supporting query files under `docs/qa/`.

This SRS should be updated when code or schema behavior changes. It intentionally does not present unsupported capabilities as complete.